FeatureLeakageLens

Interview Defense Guide — v0.2.0

Pre-training feature leakage auditor for tabular ML datasets

1. What Is FeatureLeakageLens?

FeatureLeakageLens audits tabular ML datasets for feature leakage patterns before any model is trained. It accepts a pandas DataFrame and a LeakageAuditConfig, runs six checks across three tiers, and returns a LeakageReport with per-finding PASS/WARN/FAIL/INSUFFICIENT_INPUT status.

Tier structure:

Tier	Checks	Cost	Worst status
1 — Name & Structure	Post-outcome name heuristic, ID/proxy scan	O(columns)	WARN
2 — Statistical	Target correlation, Categorical proxy, Split distribution	O(n × columns)	WARN
3 — Temporal	Temporal availability	O(n × feature_time_cols)	FAIL

Only temporal availability produces FAIL. All other checks return WARN because they surface statistical suspicions that require domain confirmation — a highly correlated feature may be a legitimate strong predictor, not a proxy.

2. Algorithmic Implementations

2.1 Post-outcome Name Heuristic

Scans column names (lowercased) against a keyword list covering common post-outcome terms: paid, received, outcome, result, decision, approved, rejected, funded, default, label, target, flag, proxy, score, prediction. Any match that is not the declared target_col triggers WARN. This is a zero-cost syntactic check — it runs before any data is read and catches the most common form of leakage: accidentally including the outcome variable under a different name.

2.2 Target Correlation Scan

Computes Pearson r between each numeric feature and the binary target. If |r| >= high_corr_threshold (default 0.95), the feature is flagged WARN. Pearson r is O(n) per feature, parameter-free, and sufficient for detecting near-perfect linear proxies — the most damaging form of leakage in practice. Nonlinear proxies (monotonic but nonlinear, or categorical encodings) are caught by separate checks. The threshold of 0.95 is deliberately high to reduce WARN fatigue from legitimate strong predictors.

2.3 Categorical Proxy Scan

For columns where n_unique / n_rows <= 0.05 (low-cardinality), computes the target rate per value. If max_rate - min_rate >= cat_proxy_threshold (default 0.50), the column is flagged WARN. A categorical feature that perfectly partitions the target — e.g., a segment column created by binning the outcome — will show a gap near 1.0. The gap measure is sensitive to class imbalance in the expected direction: a 90/10 imbalanced target produces naturally high rates in most bins, so the default threshold of 0.50 is

calibrated to fire only on extreme partitioning.

2.4 Temporal Availability

Given outcome_time_col and feature_time_cols (dict: feature → timestamp column), counts rows where feature_ts > outcome_ts. Any such row is definitively future leakage: that feature value could not have been known at prediction time. Returns FAIL (not WARN) because this is structural, not statistical — no domain context can justify a feature timestamp that postdates the outcome event. Evidence dict includes future_rows count for immediate actionability.

2.5 ID / Proxy Scan

Flags columns where n_unique / n_rows >= id_threshold (default 0.95). High-cardinality columns are potentially memorising training labels: if a customer_id appears once per row, a tree model can overfit to it perfectly. Returns WARN because some high-cardinality columns are legitimate features (e.g., hashed embeddings, zip codes in large datasets). Domain confirmation required.

2.6 Split Distribution Scan

For numeric columns: normalised absolute mean difference = |mean_train - mean_test| / pooled_std. For categorical columns: Total Variation Distance = 0.5 * sum|p_train - p_test|. Either >= distribution_shift_threshold (default 0.10) triggers WARN. Returns INSUFFICIENT_INPUT when split_col is absent — explicitly not PASS, because an unchecked distribution shift is not the same as no shift.

3. Hard Interview Questions

Q1: What is target encoding leakage? Does FeatureLeakageLens detect it?

A: Target encoding replaces a categorical value with the mean target rate in the training fold. If this is done before the train/test split — on the full dataset — the test set features contain information derived from test set labels. The model hasn't memorised the test set directly, but its features have. FeatureLeakageLens does not detect this pattern in v0: the correlation scan would flag the encoded column as highly correlated with the target (correctly), but it can't distinguish 'this is a legitimately predictive feature' from 'this was encoded using leaky full-dataset statistics'. The recommended fix is always: fit the target encoder inside the training fold of cross-validation only. Detecting this programmatically would require inspecting the feature engineering code, not just the DataFrame — a scope the current tool explicitly does not cover.

Q2: What about group leakage in k-fold cross-validation? Is that detectable here?

A: Group leakage occurs when the same entity (customer, patient, household) appears in both the training fold and the validation fold. The model sees future behaviour of the same entity during training. This produces optimistically high validation metrics. FeatureLeakageLens v0 cannot detect group leakage because the tool operates on a single DataFrame that does not carry fold assignments. Detecting it requires a GroupKFold-aware split validator that checks for entity overlap across folds. This is a documented roadmap item.

Q3: Pearson correlation misses nonlinear relationships. Why not use mutual information?

A: Mutual information is more general but introduces new problems: it requires density estimation or discretization (n_bins or k-neighbours parameter), it is sensitive to sample size in ways that vary by estimator, and it produces a scale (nats or bits) that does not have a universal threshold. Pearson r is O(n), parameter-free, and produces a universally understood scale [-1, 1]. It detects the most practically damaging leakage pattern — near-perfect linear proxy — at essentially zero cost. Nonlinear proxies are caught by the categorical proxy scan (for discrete features) and the name heuristic (for commonly named proxies). Mutual information is a documented roadmap extension.

Q4: The high_corr_threshold defaults to 0.95. Couldn't a model overfit to a feature with r = 0.80?

A: Yes, a feature with r = 0.80 could be a target proxy. The 0.95 default is a pragmatic calibration to avoid WARN fatigue: many legitimate features in well-designed datasets have r in the range 0.60–0.85. Setting the threshold at 0.95 reserves WARN for near-perfect proxies where the probability of legitimate use is low. Teams with stricter leakage tolerance can set high_corr_threshold=0.80 in config. The threshold being configurable and documented is the right design choice for a v0 tool that serves domains with very different signal levels.

Q5: What is indirect leakage? Does FeatureLeakageLens detect it?

A: Indirect leakage occurs through a causal chain: A is a post-outcome variable (unavailable at prediction time), and B is derived from A. The model is trained on B, which indirectly encodes A.

Example: default_occurred (post-outcome) → risk_tier (categorical, assigned using the default outcome) → included in features. FeatureLeakageLens would likely catch risk_tier via the categorical proxy scan (its target rate gap would be very high) and possibly via the name heuristic (if 'tier' or 'risk' triggers a keyword). However, indirect leakage through features that are downstream of post-outcome variables is not systematically detectable from a DataFrame alone without a data lineage graph. This is an explicit limitation.

Q6: Can the temporal check produce a false negative if timestamps are rounded to the day?

A: Yes. If outcome_ts and feature_ts are both stored as dates (midnight boundaries), a feature generated on the same day as the outcome will show as not future (feature_ts == outcome_ts, not strictly greater). Whether same-day features are leaky depends on the intra-day ordering of events — information FeatureLeakageLens doesn't have. The documentation should note this: teams with intra-day data should store microsecond-precision timestamps. The tool returns FAIL when there is definitive future leakage, and is silent about ambiguous same-timestamp cases rather than producing false positives.

Q7: Why use max_rate - min_rate for categorical proxy rather than Cramér's V?

A: Cramér's V measures association between a categorical feature and a categorical target and accounts for the number of categories and sample size. It is more statistically principled. The max-min gap is simpler: it directly answers 'how different is the target rate between the best and worst category?' — an intuitive quantity for explaining leakage to a domain expert. It is also more sensitive to the worst case: one very predictive category value (e.g., segment='defaulted_previously' with rate=1.0) will produce a gap of ~1.0 even if most other values are near the base rate. Cramér's V averages across categories and would dampen this signal. Cramér's V is a roadmap alternative.

Q8: What about time-series data where rows are ordered chronologically? Could the split distribution check fire spuriously?

A: Yes. Time-series data intentionally has distribution shift between train (past) and test (future) — that is by design, not a data error. The split distribution scan would flag most features in a time-series dataset as WARN. Teams using time-ordered splits should either (a) set a very high distribution_shift_threshold for their context, (b) disable the split distribution check by omitting split_col and instead use a dedicated time-series walk-forward validator. The tool's documentation should flag this: split distribution scan is most useful for random/stratified splits, not chronological splits.

Q9: FeatureLeakageLens only looks at the training DataFrame. What leakage forms does it miss entirely?

A: Several important ones. Pipeline leakage: a scaler, imputer, or encoder fit on the full dataset (including test rows) before splitting — invisible in the final DataFrame. Response variable leakage in nested cross-validation: using outer-fold labels in inner-fold feature engineering. Lookahead bias in time series: rolling window features that incorporate future values. Target-encoded features generated on the full dataset before splitting. All of these exist in the feature engineering code, not

the DataFrame, and require code inspection or runtime tracing rather than statistical auditing of the training data.

Q10: What does INSUFFICIENT_INPUT mean on the split distribution scan, and is it safe to ignore?

A: INSUFFICIENT_INPUT on the split distribution scan means split_col was not configured, so the check never ran. It is not safe to treat as PASS: the check returning INSUFFICIENT_INPUT tells you nothing about whether your train and test distributions are similar. It tells you only that you didn't ask the tool to check. The overall report status rolls up INSUFFICIENT_INPUT above PASS precisely to prevent this from looking like a clean bill of health.

4. Truth Boundary

FeatureLeakageLens does not:

• Detect pipeline leakage (scaler/encoder fit on full dataset before split)

• Detect group leakage in k-fold cross-validation (same entity in train and test folds)

• Detect target encoding leakage (target statistics computed on full dataset)

• Detect indirect / mediated leakage (features derived from post-outcome variables)

• Handle intra-day timestamp ambiguity in temporal checks

• Prove that a flagged feature caused model inflation — findings require domain confirmation

FeatureLeakageLens does:

• Check six known leakage patterns across three tiers before any model.fit() call

• Produce a documented, reproducible audit attachable to a model card or PR

• Distinguish definitive structural violations (FAIL) from statistical suspicions (WARN)

• Return evidence dicts with quantitative detail (future_rows, correlation, rate gap)

5. Files Index

File	Purpose
src/featureleakagelens/core.py	6 checks, LeakageAuditConfig, LeakageFinding, LeakageReport
tests/test_featureleakagelens.py	13 unit tests across 4 test classes
scripts/generate_demo_reports.py	Demo with synthetic leakage dataset
data/demo_leakage_dataset.csv	Demo dataset — temporal, correlation, and proxy patterns
README.md	Tiered arch diagram, About (invisible failure mode), check table
CHANGELOG.md	v0.1.0 and v0.2.0 release notes
.github/workflows/ci.yml	Python 3.10/3.11/3.12 CI matrix
.github/workflows/publish.yml	OIDC PyPI trusted publisher on release

Resume-safe claim: Built FeatureLeakageLens, a pre-training feature leakage auditor for tabular ML datasets that checks for post-outcome name heuristics, target correlation, categorical target-rate proxies, future timestamp leakage, ID/proxy columns, and train/test distribution shift, producing structured JSON/Markdown/HTML audit reports with per-finding

PASS/WARN/FAIL/INSUFFICIENT_INPUT status and explicit truth boundary.